

## Executive Summary

I investigated a security incident on the NetSysLink Production Web Server (24.49.63.79) and confirmed that an external attacker successfully broke into the system. The attacker took advantage of a weak file upload feature on the customer reviews page to upload a malicious script. By running this script, they forced the server to connect back to their computer, giving them direct access to our server's terminal commands. The attacker used this access to look around the system and copy the server's user account file (/etc/passwd).

## Overview of Affected Systems & Data

- **Affected Systems:** NetSysLink Production Web Server (24.49.63.79), specifically the Apache/Nginx web service hosting the /reviews/ web application component.
- **Compromised Accounts:** The web server service account (www-data) was directly hijacked, granting the attacker local command execution privileges within the web directory.
- **Exfiltrated Data:** The /etc/passwd system configuration file was successfully compromised and exfiltrated. While this file does not contain active password hashes, it provides a complete user account structure of the Linux system, which the attacker can leverage for targeted privilege escalation.

## Attack Timeline

The entire incident spanned just under 5 minutes (approximately 288 seconds) and unfolded across three distinct phases:

**0 seconds— Reconnaissance & Enumeration:** The attacker, operating from IP address 117.11.88.124 (Tianjin, China) using a Kali Linux Firefox user-agent, established contact with the web server (24.49.63.79). They began mapping the site's directory structure, explicitly probing /products/, /about/, and /reviews/.

**27 Seconds Later — Exploit & Payload Delivery:** The attacker targeted the review engine and executed an **HTTP POST** request to /reviews/upload.php. They successfully uploaded a malicious PHP web shell disguised with a double extension: image.jpg.php.

**84.15 Seconds — Foothold & Execution:** The attacker navigated directly to the stored script path at /reviews/uploads/image.jpg.php via an **HTTP GET** request. This triggered the payload, causing the server to initiate an outbound TCP handshake (**SYN**) back to the attacker's infrastructure on port **8080**, successfully spawning an interactive reverse shell.

**191.37 Seconds — Data Exfiltration:** Operating through the reverse shell as the www-data user, the attacker queried system information (`uname -a`) and ran `cat /etc/passwd` to view local system accounts. The attacker then executed a curl POST request to transmit the contents of `/etc/passwd` out of the network to their command-and-control server over port 443.

## Step-by-step documentation of investigation and packet analysis

### 1 Identify the Geographical Origin of the Attack

From which city did the attack originate?

I identified the attacker IP from the PCAP file which I opened using Wireshark.

From the Wireshark I went to STATISTICS > CONVERSATIONS > IPv4

I isolated the external IP address responsible for the traffic: **117.11.88.124**.

I ran this IP through **IPinfo.io** to map its origin, which points directly to **Tianjin, China**. While the data physically traces back to Tianjin.

|         |                       |
|---------|-----------------------|
| City    | Tianjin               |
| State   | Tianjin               |
| Country | <a href="#">China</a> |

Tool Used: **ipinfo.io**

### 2 Determine the Attacker's User-Agent

What was the attacker's User-Agent string, and what does it tell us about the tool or browser used?

The attacker User-Agent string is `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0\r\n`

I got the user-agent string using this wireshark filter `ip.src == 117.11.88.124` and http

| No. | Time      | Source        | Destination | Protocol | Length | Type | Info                                        |
|-----|-----------|---------------|-------------|----------|--------|------|---------------------------------------------|
| 8   | 0.037487  | 117.11.88.124 | 24.49.63.79 | HTTP     | 356    | IPv4 | GET /favicon.ico HTTP/1.1                   |
| 11  | 4.435305  | 117.11.88.124 | 24.49.63.79 | HTTP     | 444    | IPv4 | GET /products/ HTTP/1.1                     |
| 14  | 4.458038  | 117.11.88.124 | 24.49.63.79 | HTTP     | 382    | IPv4 | GET /products/images/product1.jpg HTTP/1.1  |
| 19  | 4.458504  | 117.11.88.124 | 24.49.63.79 | HTTP     | 382    | IPv4 | GET /products/images/product2.jpg HTTP/1.1  |
| 33  | 12.739450 | 117.11.88.124 | 24.49.63.79 | HTTP     | 450    | IPv4 | GET /about/ HTTP/1.1                        |
| 43  | 18.514912 | 117.11.88.124 | 24.49.63.79 | HTTP     | 449    | IPv4 | GET /reviews/ HTTP/1.1                      |
| 53  | 26.922481 | 117.11.88.124 | 24.49.63.79 | HTTP     | 1304   | IPv4 | POST /reviews/upload.php HTTP/1.1 (applicat |
| 63  | 49.758143 | 117.11.88.124 | 24.49.63.79 | HTTP     | 1302   | IPv4 | POST /reviews/upload.php HTTP/1.1 (applicat |
| 73  | 57.538074 | 117.11.88.124 | 24.49.63.79 | HTTP     | 416    | IPv4 | GET /admin/uploads HTTP/1.1                 |
| 83  | 63.058836 | 117.11.88.124 | 24.49.63.79 | HTTP     | 410    | IPv4 | GET /uploads HTTP/1.1                       |
| 93  | 69.755241 | 117.11.88.124 | 24.49.63.79 | HTTP     | 409    | IPv4 | GET /admin/ HTTP/1.1                        |
| 103 | 75.291187 | 117.11.88.124 | 24.49.63.79 | HTTP     | 418    | IPv4 | GET /reviews/uploads HTTP/1.1               |
| 107 | 75.207010 | 117.11.88.124 | 24.49.63.79 | HTTP     | 419    | IPv4 | GET /reviews/uploads/ HTTP/1.1              |

```
> Frame 11: Packet, 444 bytes on wire (3552 bits), 444 bytes captured (3552 bits)
> Ethernet II, Src: VMware_c0:00:09 (00:50:56:c0:00:09), Dst: VMware_61:97:cd (00:0c:29:61:97:cd)
> Internet Protocol Version 4, Src: 117.11.88.124, Dst: 24.49.63.79
> Transmission Control Protocol, Src Port: 43848, Dst Port: 80, Seq: 628, Ack: 1222, Len: 378
> Hypertext Transfer Protocol
  > GET /products/ HTTP/1.1\r\n
    Host: shoporama.com\r\n
    User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0\r\n
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8\r\n
    Accept-Language: en-US,en;q=0.5\r\n
    Accept-Encoding: gzip, deflate\r\n
```

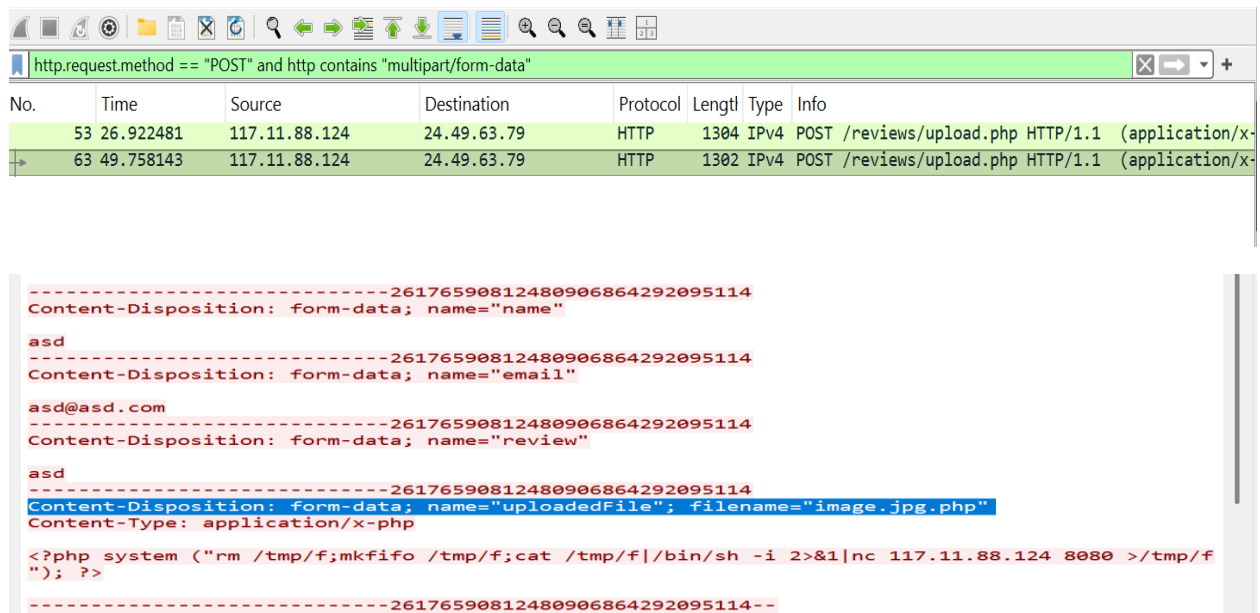
### 3 Identify the Malicious Web Shell

What is the name of the malicious web shell that was successfully uploaded?

The name of the malicious web shell is `image.jpg.php`

The attacker successfully uploaded a malicious web shell named `image.jpg.php`. They used a classic double-extension tactic (`.jpg.php`) to trick the server's upload filter. Security filters often only check for the presence of an allowed extension like `.jpg` to verify an image; by appending `.php` to the very end, the attacker ensured that the web server would still treat and execute the file as a functional PHP script once it was stored.

I used this wireshark filter `http.request.method == "POST"` and http contains "multipart/form-data" to carry out the task. After using the filter to isolate the packets (53 & 63) that contain the malicious file I right click on the packet > Click follow > then Click HTTP Stream to view the file name.



#### 4 Discover the Directory Used for File Uploads

Which directory on the server was used to store uploaded files?

The directory used to store the Uploaded files is `/reviews/uploads/`

To determine where the web server stores uploaded items, I analyzed the attacker's HTTP GET requests following their file upload attempts.

According to the packet logs (specifically Packet 138), the attacker immediately attempted to access and execute their script by navigating to `/reviews/uploads/image.jpg.php`. This confirms that the web application's review function routes all user-submitted uploads directly into the `/reviews/uploads/` directory on the server file system.

I used this Wireshark filter `ip.src == 117.11.88.124 and http.request.method == "GET"` to carry out the task.

| ip.src == 117.11.88.124 and http.request.method == "GET" |            |               |             |          |        |      |                                             |
|----------------------------------------------------------|------------|---------------|-------------|----------|--------|------|---------------------------------------------|
| No.                                                      | Time       | Source        | Destination | Protocol | Length | Type | Info                                        |
| 73                                                       | 57.538074  | 117.11.88.124 | 24.49.63.79 | HTTP     | 416    | IPv4 | GET /admin/uploads HTTP/1.1                 |
| 83                                                       | 63.058836  | 117.11.88.124 | 24.49.63.79 | HTTP     | 410    | IPv4 | GET /uploads HTTP/1.1                       |
| 93                                                       | 69.755241  | 117.11.88.124 | 24.49.63.79 | HTTP     | 409    | IPv4 | GET /admin/ HTTP/1.1                        |
| 103                                                      | 75.201187  | 117.11.88.124 | 24.49.63.79 | HTTP     | 418    | IPv4 | GET /reviews/uploads HTTP/1.1               |
| 107                                                      | 75.207010  | 117.11.88.124 | 24.49.63.79 | HTTP     | 419    | IPv4 | GET /reviews/uploads/ HTTP/1.1              |
| 109                                                      | 75.228143  | 117.11.88.124 | 24.49.63.79 | HTTP     | 376    | IPv4 | GET /icons/blank.gif HTTP/1.1               |
| 114                                                      | 75.228890  | 117.11.88.124 | 24.49.63.79 | HTTP     | 375    | IPv4 | GET /icons/back.gif HTTP/1.1                |
| 121                                                      | 75.229218  | 117.11.88.124 | 24.49.63.79 | HTTP     | 377    | IPv4 | GET /icons/image2.gif HTTP/1.1              |
| 138                                                      | 84.150547  | 117.11.88.124 | 24.49.63.79 | HTTP     | 480    | IPv4 | GET /reviews/uploads/image.jpg.php HTTP/1.1 |
| 326                                                      | 288.389226 | 117.11.88.124 | 24.49.63.79 | HTTP     | 470    | IPv4 | GET /reviews/uploads/ HTTP/1.1              |
| 330                                                      | 288.400569 | 117.11.88.124 | 24.49.63.79 | HTTP     | 427    | IPv4 | GET /icons/blank.gif HTTP/1.1               |
| 335                                                      | 288.401559 | 117.11.88.124 | 24.49.63.79 | HTTP     | 426    | IPv4 | GET /icons/back.gif HTTP/1.1                |
| 340                                                      | 288.401886 | 117.11.88.124 | 24.49.63.79 | HTTP     | 428    | IPv4 | GET /icons/image2.gif HTTP/1.1              |

## 5 Determine the Port Used for Outbound Communication

Which outbound port did the malicious web shell use to contact the attacker's machine?

The Outbound port is 8080

Following the execution of the web shell, the server immediately initiated an outbound connection back to the attacker's machine. Forensic analysis of Packet 140 captures the server opening a raw TCP connection ([SYN]) from a local random ephemeral port (54448) directly to destination port **8080** on the attacker's host. This confirms the deployment of a reverse shell on a common alternative port (8080), allowing the attacker to successfully bypass standard ingress firewall boundaries.

I used this wireshark filter `ip.src == 24.49.63.79 and ip.dst == 117.11.88.124 and tcp and not tcp.port == 80`

| ip.src == 24.49.63.79 and ip.dst == 117.11.88.124 and tcp and not tcp.port == 80 |            |             |               |          |        |      |                                                        |
|----------------------------------------------------------------------------------|------------|-------------|---------------|----------|--------|------|--------------------------------------------------------|
| No.                                                                              | Time       | Source      | Destination   | Protocol | Length | Type | Info                                                   |
| 140                                                                              | 84.153968  | 24.49.63.79 | 117.11.88.124 | TCP      | 74     | IPv4 | 54448 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS=          |
| 142                                                                              | 84.154497  | 24.49.63.79 | 117.11.88.124 | TCP      | 66     | IPv4 | 54448 → 8080 [ACK] Seq=1 Ack=1 Win=64256 Len=0         |
| 143                                                                              | 84.154602  | 24.49.63.79 | 117.11.88.124 | TCP      | 121    | IPv4 | 54448 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=64256 Len=0    |
| 146                                                                              | 88.912162  | 24.49.63.79 | 117.11.88.124 | TCP      | 66     | IPv4 | 54448 → 8080 [ACK] Seq=56 Ack=8 Win=64256 Len=0        |
| 147                                                                              | 88.912892  | 24.49.63.79 | 117.11.88.124 | TCP      | 75     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=56 Ack=8 Win=64256 Len=0   |
| 149                                                                              | 88.913086  | 24.49.63.79 | 117.11.88.124 | TCP      | 68     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=65 Ack=8 Win=64256 Len=0   |
| 152                                                                              | 93.976778  | 24.49.63.79 | 117.11.88.124 | TCP      | 208    | IPv4 | 54448 → 8080 [PSH, ACK] Seq=67 Ack=17 Win=64256 Len=0  |
| 154                                                                              | 93.976974  | 24.49.63.79 | 117.11.88.124 | TCP      | 68     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=209 Ack=17 Win=64256 Len=0 |
| 163                                                                              | 117.909424 | 24.49.63.79 | 117.11.88.124 | TCP      | 98     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=211 Ack=21 Win=64256 Len=0 |
| 166                                                                              | 124.265867 | 24.49.63.79 | 117.11.88.124 | TCP      | 73     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=243 Ack=30 Win=64256 Len=0 |
| 168                                                                              | 124.266042 | 24.49.63.79 | 117.11.88.124 | TCP      | 68     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=250 Ack=30 Win=64256 Len=0 |

## 6 Identify the File Targeted for Exfiltration

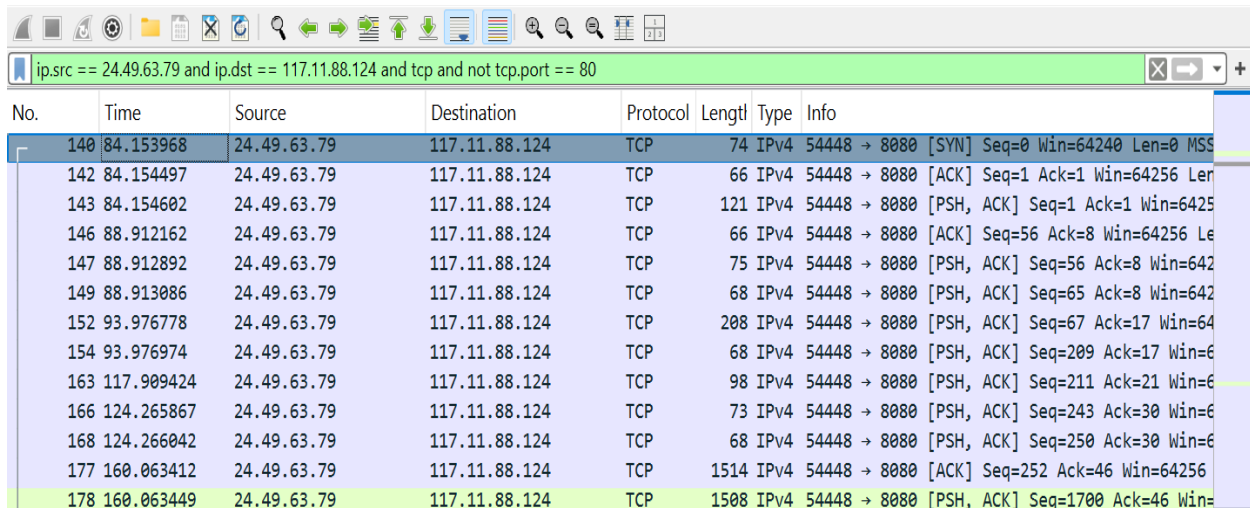
What file did the attacker attempt to extract from the server?

The file attacker attempt to extract in the is `/etc/passwd`

When the attacker established interactive command-line access through the reverse shell on port 8080, they ran a sequence of system discovery commands (`whoami`, `uname -a`, and `pwd`) to orient themselves.

After confirming they were operating with `www-data` web server privileges, the attacker targeted the core Linux user configuration file by executing: `cat /etc/passwd`.

Using this wireshark filter `ip.src == 24.49.63.79 and ip.dst == 117.11.88.124 and tcp and not tcp.port == 80`, I Right Clicked packet **140** > follow > tcp stream then identified the file that was targeted which is `/etc/passwd`



| No. | Time       | Source      | Destination   | Protocol | Length | Type | Info                                          |
|-----|------------|-------------|---------------|----------|--------|------|-----------------------------------------------|
| 140 | 84.153968  | 24.49.63.79 | 117.11.88.124 | TCP      | 74     | IPv4 | 54448 → 8080 [SYN] Seq=0 Win=64240 Len=0 MSS= |
| 142 | 84.154497  | 24.49.63.79 | 117.11.88.124 | TCP      | 66     | IPv4 | 54448 → 8080 [ACK] Seq=1 Ack=1 Win=64256 Len= |
| 143 | 84.154602  | 24.49.63.79 | 117.11.88.124 | TCP      | 121    | IPv4 | 54448 → 8080 [PSH, ACK] Seq=1 Ack=1 Win=6425  |
| 146 | 88.912162  | 24.49.63.79 | 117.11.88.124 | TCP      | 66     | IPv4 | 54448 → 8080 [ACK] Seq=56 Ack=8 Win=64256 Le  |
| 147 | 88.912892  | 24.49.63.79 | 117.11.88.124 | TCP      | 75     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=56 Ack=8 Win=642  |
| 149 | 88.913086  | 24.49.63.79 | 117.11.88.124 | TCP      | 68     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=65 Ack=8 Win=642  |
| 152 | 93.976778  | 24.49.63.79 | 117.11.88.124 | TCP      | 208    | IPv4 | 54448 → 8080 [PSH, ACK] Seq=67 Ack=17 Win=64  |
| 154 | 93.976974  | 24.49.63.79 | 117.11.88.124 | TCP      | 68     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=209 Ack=17 Win=6  |
| 163 | 117.909424 | 24.49.63.79 | 117.11.88.124 | TCP      | 98     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=211 Ack=21 Win=6  |
| 166 | 124.265867 | 24.49.63.79 | 117.11.88.124 | TCP      | 73     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=243 Ack=30 Win=6  |
| 168 | 124.266042 | 24.49.63.79 | 117.11.88.124 | TCP      | 68     | IPv4 | 54448 → 8080 [PSH, ACK] Seq=250 Ack=30 Win=6  |
| 177 | 160.063412 | 24.49.63.79 | 117.11.88.124 | TCP      | 1514   | IPv4 | 54448 → 8080 [ACK] Seq=252 Ack=46 Win=64256   |
| 178 | 160.063449 | 24.49.63.79 | 117.11.88.124 | TCP      | 1508   | IPv4 | 54448 → 8080 [PSH, ACK] Seq=1700 Ack=46 Win=  |

```
/bin/sh: 0: can't access tty; job control turned off
$
whoami
www-data
$
uname -a
Linux ubuntu-virtual-machine 6.2.0-37-generic #38~22.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Nov 2
18:01:13 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
$
pwd
/var/www/html/reviews/uploads
$
ls /home
ubuntu
$
cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
```

## Recommendations

To remediate this active compromise and harden the infrastructure against future file-upload exploits, implement the following defensive controls:

### Short-Term (Immediate Containment)

1. **Terminate Active Sessions:** Kill the rogue outbound TCP process communicating over port 8080 and block the attacker's IP address (117.11.88.124) at the edge perimeter firewall.
2. **Payload Elimination:** Delete the `image.jpg.php` file from the `/var/www/html/reviews/uploads/` directory.
3. **Precautionary Credential Rotation:** Force a password reset for all system accounts enumerated in the leaked `/etc/passwd` file.

### Long-Term Security Fixes

#### 1. Secure the File Upload Feature

- Make the server automatically rename every uploaded file to a random string (like `upload_98732.jpg`) and strip out any extra extensions.

#### 2. Stop Scripts from Running in Upload Folders

- Add a rule to the web server settings (Apache or Nginx) that blocks and denies any request to run `.php` files inside the uploads folder.

#### 3. Block Unapproved Outbound Connections

- Configure the firewall to block all outbound traffic leaving the web server, unless it is going to an approved list of safe destinations (like official update servers or internal databases). This completely breaks backdoor connection

## Conclusion

This incident shows how a single weak feature can expose an entire server. The attacker used a flawed file upload page on the website to drop a malicious script, open a backdoor, and access the server's command line.

I successfully tracked every step of the attack from the initial connection down to the exact moment they copied the system user file (/etc/passwd).

By putting my recommended fixes into action specifically fixing the file upload security and blocking unapproved outgoing connections at the firewall we will close these gaps and stop this type of attack from happening again.

Report Prepared by:  
JOSEPH EMEKA ANI